CV32RT: Enabling Fast Interrupt and Context Switching for RISC-V Microcontrollers

Robert Balas, *Student Member, IEEE*, Alessandro Ottaviano, *Student Member, IEEE*, and Luca Benini, *Fellow, IEEE*

***Abstract*—Processors using the open RISC-V ISA are finding increasing adoption in the embedded world. Many embedded use cases have real-time constraints and require flexible, predictable, and fast reactive handling of incoming events. However, RISC-V processors are still lagging in this area compared to more mature proprietary architectures, such as ARM Cortex-M and TriCore, which have been tuned for years. The default interrupt controller standardized by RISC-V, the Core Local Interruptor (CLINT), lacks configurability in prioritization and preemption of interrupts. The RISC-V Core Local Interrupt Controller (CLIC) specification addresses this concern by enabling preemptible, low-latency vectored interrupts while also envisioning optional extensions to improve interrupt latency. In this work, we implement a CLIC for the CV32E40P, an industrially supported open-source 32-bit MCU-class RISC-V core, and enhance it with `fastirq`: a custom extension that provides interrupt latency as low as 6 cycles. We call CV32RT our enhanced core. To the best of our knowledge, CV32RT is the first fully open-source RV32 core with competitive interrupt-handling features compared to the Arm Cortex-M series and TriCore. The proposed extensions are also demonstrated to improve task context switching in real-time operating systems.**

***Index Terms*—RISC-V, real-time, interrupt latency, context switching, MCU, embedded**

I. Introduction

SEVERAL markets, from automotive to aerospace and robotics, rely on real-time hardware and software [1]. For example, the automotive industry employs hundreds of electronic control units (ECUs) for real-time applications, such as electronic engine control, gearbox control, cruise control, anti-lock brake systems, and many other tasks.

General-purpose operating systems (GPOSs) are typically tuned for average throughput rather than real-time requirements imposed by such scenarios. For example, the development of Linux, a popular open-source GPOS kernel, is focused on average performance, making it less suitable to be used for real-time applications [2].

Albeit extensions and modifications which aim at improving determinism and latencies of critical operations in Linux have been proposed and implemented [2]–[5], they do not guarantee strict bounds on maximum latencies of operations and lack industry-grade maturity to be employed in hard real-time scenarios.

R. Balas, A. Ottaviano, and L. Benini are with the Integrated Systems Laboratory (IIS), ETH Zurich, Switzerland.
E-mail: {balasr, aottaviano,lbenini}@iis.ee.ethz.ch
L. Benini is also with the Department of Electrical, Electronic and Information Engineering (DEI), University of Bologna, Bologna, Italy.

This work was supported in part through the TRISTAN (101095947) project that received funding from the HORIZON KDT-JU programme.

Real-time operating systems (RTOSs) kernels are special-purpose Operating Systems (OSs) designed to provide real-time guarantees, such as task scheduling according to a given expected completion deadline and deterministic latencies of various operations [6].

The RTOS' scheduler might add a significant overhead due to the combined effect of both the *context switches* required to handle the transition from a foreground to a background task and the amount of time elapsed from the source event that causes the preemption and the first instruction of the awakened task (known as *interrupt latency* [7]), thus increasing the worst-case execution time (WCET) [2], [8], [9].

The cost of saving and restoring the task state during a context switch is a significant concern as it remains relatively high. For instance, the process state that needs to be saved on a context switch includes the program counter, register files, status registers, address space mapping, etc. Therefore, a significant number of memory access operations need to be performed to *store* the state of the preempted task and *restore* the state of the new task to be executed [10]. Long context switch times reduce available *task utilization* and the minimum viable switching granularity.

Besides the task's context switching, a switch into an interrupt context from normal program execution happens each time an asynchronous event is triggered from, e.g., an I/O peripheral device. Behnke et al. [11] provide a meaningful example of the impact on *interrupt latency* of network loads in time-critical microcontroller units (MCUs), where a high overhead generated by the receiving of packets can be seen in continuous floods as well as short transmission bursts, reaching up to 50% task *lateness* — the additional time a task takes to finish than its deadline allows, which will start to accumulate over iterations from the critical network load — increase per packet per second. Furthermore, hardware-induced interrupt latency is only one part of the problem: software-induced interrupt latency, introduced by the GPOS/RTOS scheduler and the user code, primarily impacts the capability of the system to provide timely responses to asynchronous events.

Low interrupt latency and context switch time are crucial metrics for a wide range of platforms ranging from commodity MCU-class embedded systems to more advanced and complex application-class mixed criticality systems (MCSs), where time/safety-critical and non-critical applications coexist on different isolated partitions of the same hardware platform [12].

Response and *context switch* time minimization thus become a challenge to be tackled at the hardware (HW) / software (SW) interface, where SW programming techniques and HW

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

interrupt controller architectures can cooperate to ensure minimal response time [7].

Although commercial vendors and IP providers offer such features as in-house solutions, they are often proprietary and tightly coupled with the vendor's instruction set architecture (ISA), target hardware family, and associated software stack. On the other hand, in the last decade, the ever-growing RISC-V ecosystem [13] has been offering a modular, free, and open-source ISA which is rapidly becoming the *de facto lingua franca* of computing.

Nevertheless, RISC-V support for fast interrupt and context switch handling is still not mature enough to compete with incumbent proprietary architectures, as it lacks flexible interrupt prioritization, preemption mechanisms, and low interrupt latency. For this reason, the RISC-V community has been developing an extension to the *Privileged* specifications [14] with the proposal of the RISC-V Core-Local Interrupt Controller (CLIC) [15], currently under ratification by the community, to handle such real-time scenarios.

While the modular RISC-V ISA enables developing orthogonal *custom extensions*, to the best of the authors' knowledge, there are few published works that (i) tackle the problem of minimizing *interrupt latency* and *context switch* time from a holistic (HW and SW) viewpoint for RISC-V, (ii) try to close the gap with more established proprietary solutions, thereby promoting RISC-V as a valuable candidate for time- and safety-critical application domains such as automotive and aerospace, and (iii) provide an open-source solution to be shared with the community.

To address these open challenges, we propose CV32RT, a 32-bit RISC-V core that extends the interrupt handling capabilities of CV32E40P [16], [17], an industrially supported open-source core, to achieve best-in-class interrupt latency and fast context switching against commercial off-the-shelf (COTS) processor vendors, paving the road for RISC-V architectures in time-critical systems.

In particular, this paper makes the following contributions:

- We replace the CV32E40P interrupt controller with an implementation of the RISC-V CLIC specification, which provides the core with key features such as prioritization by level and priority, selective hardware vectoring, and non-nested interrupt optimization (*tail-chaining* through the `xnxti` [15] CSR) directly as RISC-V standard extension (Section III-A). We make the implementation available under a permissive open-source license [1].
- We design a *fast interrupt extension* (`fastirq`) to accelerate both nested (Section III-A) and non-nested (Section III-D) interrupt case scenarios. `fastirq` reduces *interrupt latency* by hiding the latency through memory banks and a background saving mechanism. The same mechanism allows one also to accelerate *context switching* through HW/SW cooperation. Furthermore, we propose `early mret` (`emret`), a novel instruction that further optimizes tail-chaining compared to the baseline strategy proposed in the CLIC standard (i.e., `xnxti`) and its enhancement from [18] (`jalxnxti`).
- We integrate CV32RT within an open-source system on chip (SoC) [19] [2]. We evaluate CV32RT interrupt handling capabilities while showing the negligible area overhead introduced by the extension in a modern technology node against its performance benefits (Section IV).
- Finally, we compare CV32RT with leading COTS systems in both nested and non-nested interrupt scenarios (Section V). We show that the proposed solution promotes RISC-V as a competitive candidate for building the next generation of time-critical systems.

II. Background

We give a brief overview of the full-system platform used to design and implement the proposed interrupt extension in Section II-A, motivate and explain the relevant target metrics in Section II-B3, and describe the current status of interrupt handling in RISC-V in Section II-C.

A. RISC-V system platform

We rely on the CV32E40P core [16] — abbreviated to CV32 in this paper — an open-source, industry-grade, 32-bit, in-order, four-stage RISC-V core, as the basis for implementing our extensions. This core is embedded in ControlPULP [19], a SoC specialized in running real-time workloads. The system contains a CV32 manager core, a programmable accelerator subsystem consisting of 8 CV32 cores, and a set of standard peripherals such as QSPI, I2C, and UART. The manager core is responsible for scheduling tasks, communicating with the peripherals, offloading tasks to the accelerator subsystem, and being responsive to asynchronous external events, e.g., interrupts.

ControlPULP hosts a set of scratchpad memories (SPMs) that guarantee *single-cycle access time* from the CV32 manager core. This design choice enables deterministic memory access latency for both data load, store, and instruction fetch, bounding the worst-case latency when handling unpredictable events.

Applications run on top of FreeRTOS [20], an open-source, priority-based preemptive RTOS, in the manager core from where tasks are scheduled and run.

B. Interrupts

1) Level- and edge-triggered interrupts: Interrupt sources can signal interrupts either through a level change of the interrupt line, called a *edge-triggered interrupt*, or by the logic level itself, a *level-triggered interrupt*. While the latter requires the receiving side of the interrupt to clear the source often through accessing appropriate hardware registers, the former is unidirectional notification without confirmation. This results in faster processing but may lead to dropping interrupt requests accidentally.

[1] https://github.com/pulp-platform/clic

[2] https://github.com/pulp-platform/control-pulp

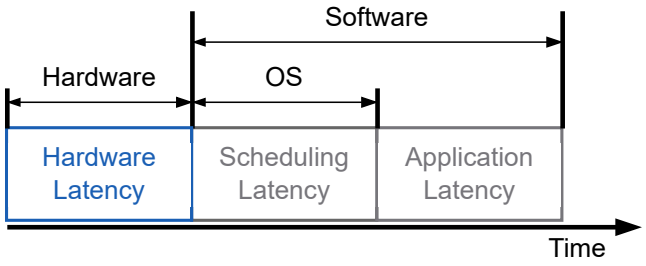


Fig. 1. Interrupt latency breakdown into software and hardware dependent contributions.

2) Vectored and direct interrupts: Interrupts are asynchronous events that alter the *normal program order execution* and require a switch to a different context to handle the event. A processor supports *vectored* interrupts when each interrupt traps to a specific interrupt service routine (ISR) according to an *interrupt vector table*, granting fast interrupt response at the cost of increased code size. Conversely, *non-vectored* or *direct* interrupts trap to a common ISR. The latter approach trades off code size for a slower interrupt response due to the overhead of resolving the cause of the interruption and jumping to the correct ISR.

3) Interrupt latency: Multiple sources determine the interrupt latency in a system: the underlying hardware, the scheduler/OS, and the application running on top as detailed in Figure 1.

In this paper, we focus on minimizing the latency imposed by the hardware; thus, whenever we refer to interrupt latency, we mean its hardware-contributed part. Interrupt latency is defined as the time it takes from an interrupt edge arriving at the hardware, usually the interrupt controller, to the execution of the first instruction of the corresponding interrupt handler routine. To make a fair comparison between software-based and more hardware-oriented interrupt solutions, we have to delineate what exactly constitutes the first instruction of the interrupt handler. We count as the first instruction the one after all necessary interrupt context has been saved on the stack to be able to call a *function*. What exactly entails a function call is dictated by the used application binary interface (ABI) or, more precisely, its calling convention. Note that when the interrupt handler and interrupt context saving code can be interleaved, some of the context saving code can be removed as it might be redundant.

4) Context switching time: While interrupt latency is representative of the responsiveness of the hardware, scheduler, and application in arbitrating and servicing potentially asynchronous events with different priorities, context switching time refers to the responsiveness of the architecture in swapping from one execution context to another. The execution context is dependent on the OS being used but usually consists of the *state* of the architectural registers of the ISA and the chosen ABI.

5) Nested and non-nested interrupts: Preemption refers to an event, such as an interrupt request, temporarily interrupting a current task with the purpose of resuming its execution later.

The simplest case for preemption occurs with *non-nested interrupt handlers*, where interrupts are globally disabled during the execution of an ISR. In the case of level/priority interrupt schemes, this means that (i) if the level/priority arbitration is SW-driven, e.g., with priority simple/standard interrupt handlers, the highest priority interrupt identification code is not executed, and (ii) if the level/priority arbitration logic is designed within the interrupt controller, the highest level/priority interrupt is pending but disabled, hence not propagated to the core. This scenario is not ideal for real-time and complex embedded systems, as interrupts are served sequentially. A high-priority interrupt has to wait for a lower-priority interrupt to finish.

A more complex case for preemption occurs with *nested interrupt handlers* to handle the case of multiple interrupts at a time. In this scenario, interrupts are globally enabled within the scope of an executing ISR. The ISR need to care designed to ensure the they are reentrant. With a level/priority interrupt scheme (either SW or HW driven), this introduces additional masking of incoming interrupts of equal or lower level/priority than the executing ISR, and sometimes larger than a configurable level/priority threshold. The nesting allows higher priority interrupts to preempt a current lower priority ISR executing.

6) Redundant context restore with non-nested and masked back-to-back interrupts: We indicate as *back-to-back* interrupts served one after the other because preemption cannot occur. As detailed above, this is the case of interrupts globally masked in the non-nested scenario or masked according to level/priority schemes when nesting is allowed. In this situation, the active interrupt handler's context must be restored to be interrupted and saved again before handling the new interrupt event, causing a *redundant context restore*. These redundant context restore sequences negatively impact interrupt latency on higher interrupt loads [21]. Section III-D explores the optimizations implemented in this work to address this scenario.

C. Interrupt handling in RISC-V

1) CLINT and PLIC: We distinguish between *core local interrupt controllers*, local to each hardware thread (HART), and *platform level interrupt controllers*, centralized interrupt controllers capable of addressing multiple HARTs. In the RISC-V ecosystem, the privileged specification [14] defines a simple interrupt scheme with a set of timer and inter-processor interrupts. We refer to it as the Core-Local Interruptor (CLINT). For 32-bit cores, it defines a fixed priority interrupt scheme with 16 predefined or reserved and 16 implementation-defined interrupts that can be optionally vectored. The CLINT itself only supports prioritization of interrupts based on privilege mode. This is insufficient for the embedded and real-time domain, and attempts to handle a more complex scheme would require software emulation, which incurs untenable interrupt latencies. The interrupt handling is mostly software-based. To increase the number of custom interrupts, a Platform-Level Interrupt Controller (PLIC) [22] can be attached to the CLINT. It can route a flexible (at design time) number of interrupts to one or more targets. Each interrupt can be assigned a priority, and each target can select a threshold below which

TABLE I
NESTED INTERRUPT PREEMPTION SCHEME ACCORDING TO RISC-V CLIC. WE ASSUME AN INTERRUPT REQUEST `IRQ2` IS BEING SERVICED WHILE ANOTHER REQUEST `IRQ1` IS ENABLED AND PENDING. PREEMPTION IS REGULATED BY PRIVILEGE MODE (*vertical* INTERRUPTS) AND INTERRUPT LEVEL WHEN THE PRIVILEGE MODE IS THE SAME (*horizontal* INTERRUPTS).

Nested interrupt behaviour	Condition	CLINT preemption?	CLIC preemption?
Vertical	**priv1** $>$ **priv2**	✓	✓
Horizontal	(**priv1** $==$ **priv2**) $\wedge$ ((**l1** $>$ **thresh**) $\wedge$ (**l1** $>$ **l2**))	✗	✓

interrupts are disabled. This scheme allows interrupts to be divided according to their priorities.

2) CLIC: The CLIC [15] addresses these limitations by allowing interrupts to be prioritized by so-called *levels* and *priorities*. Interrupt selection is driven by the CLIC in hardware (Section III-B), which propagates the highest level, highest priority pending interrupt to the core's interface. Compared to standard RISC-V privileged specifications, pending and enabled interrupts are further selectively masked according to a *threshold value* representing an interrupt level, configured through a Control and Status Register (CSR). Interrupts that are enabled, pending, and have a level below the threshold are masked, while others can be propagated. This feature is beneficial for RTOSs that allow certain types of critical interrupts to preempt execution still.

Consider two interrupts requests `irq1` and `irq2` with interrupt privilege modes, levels and priorities (`priv1`, `l1`, `p1`) and (`priv2`, `l2` and `p2`) respectively. Following the terminology introduced in Section II-B and by the CLIC, we will refer to nested interrupts fired from different privilege modes as *vertical* interrupts and to nested interrupts fired from the same privilege mode as *horizontal* interrupts throughout this paper. Table I shows preemption conditions of two nested interrupts `irq2` and `irq1` according to the CLIC specification. We consider an active interrupt handler servicing `irq2` while `irq1` is enabled, pending, and with a higher level than both the threshold and `irq2`. Interrupt priority is considered a tie-breaker for the case of multiple horizontal interrupts with the same level and does not cause preemption, i.e., pending interrupts are serviced according to increasing priority. Analogously, the case where multiple horizontal interrupts have equal levels and priorities results in the CLIC selecting the highest numbered interrupt (identification number `id`) [15], which is an arbitrary assignment at design time.

Lastly, the CLIC specification addresses the case of *redundant context restore* (Section II-B6) by introducing `xnxti`, a CSR short for *Next Interrupt Handler Address and Interrupt-Enable CSRs* meant for use with non-vectored interrupts. Reading from this CSR, while the core is within an active handler, allows to fast-track interrupts that arrive late or to avoid redundant context save/restore by running through pending interrupts back-to-back.

III. ARCHITECTURE

Entering an interrupt context or performing a context switch requires the hardware to store enough information to resume operation correctly after returning from the aforementioned context. This needs to be done as fast as possible. From a high-level point of view, we can effectively improve interrupt latency and context switch times by:

- Controlling the amount of state that needs to be preserved to enter and leave an interrupt context.
- Increasing the bandwidth and decreasing the latency to memory.
- Relying on *latency-hiding* techniques that defer the effective saving of the state to a later point in time.

In the following, we detail the architectural features and HW/SW codesign of CV32RT (Section III-A Section III-B, and Section III-C), and then describe how the proposed architecture tackles typical case scenarios (Section III-D).

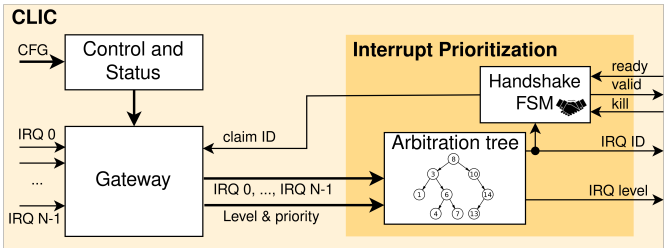


Fig. 2. CLIC architecture overview. Interrupts arrive at the gateway, where they are combined with programmable configuration information about each interrupt line consisting of level, priority, enable status, and sensitivity (level/edge). Enabled interrupts and their level and priority information are then further sent to prioritization logic which uses a binary arbitration tree to select the highest-level interrupt. The interrupt is then presented to the core with a handshake-based interface. The additional kill signal is there to allow for a handshake to restart so that a potentially more important interrupt can be presented to the core.

A. CV32RT overview

We start with the CV32 core as the baseline, whose native CLINT interrupt controller we replace with the CLIC (hereafter, CV32RT$^{\texttt{CLIC}}$), and then introduce `fastirq` into the core microarchitecture, the extension proposed in this work (hereafter, CV32RT$^{\texttt{fastirq}}$). CV32RT$^{\texttt{fastirq}}$ optimizes interrupt latency in the non-nested interrupt case by combining bank switching and the nested interrupt case with an automatic context-saving mechanism in the background. The background saving mechanism updates the stack pointer and stores the bank-switched contents in memory while the execution of the core proceeds in parallel.

We describe each of these changes in detail below.

B. CV32RT^{CLIC}: CLIC fast interrupt controller

We implement the CLIC interrupt controller in CV32RT$^{\texttt{CLIC}}$ [15] according to the draft specification which is to be included in the RISC-V Privileged Specification [14]. Figure 2 provides an overview of the design. Incoming interrupts are filtered with a `Gateway` module that decides whether there is a pending and enabled request for each interrupt source *i* (`IRQ i`).

Assuming *n* interrupt sources, selecting the interrupt with maximum level and priority is implemented with three binary

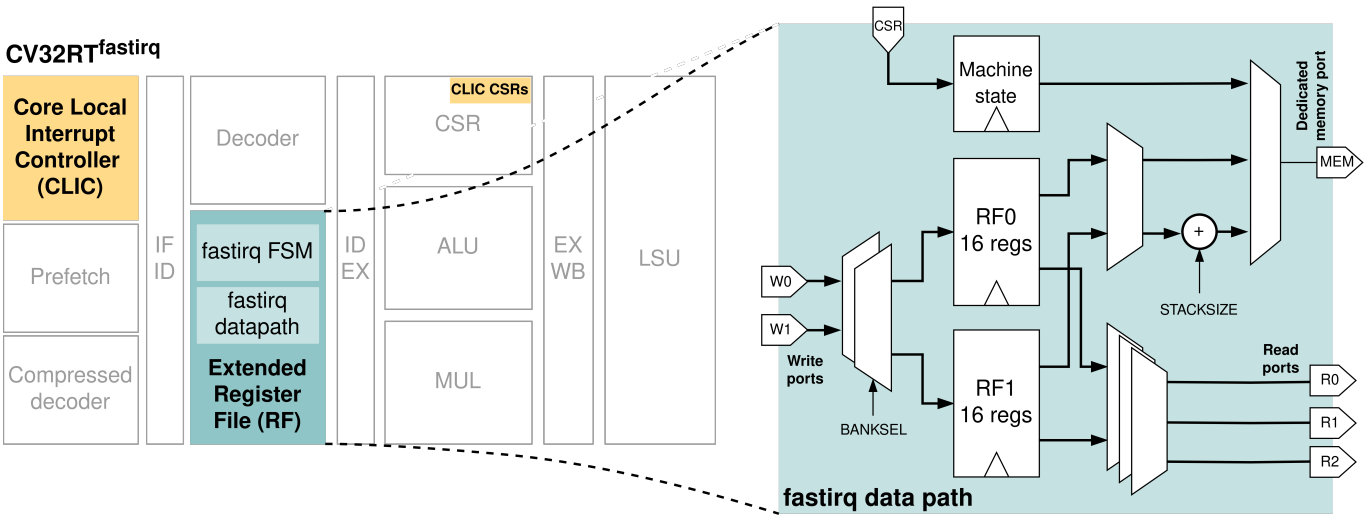


Fig. 3. Overview of the fast interrupt architecture. The cv32's register file is extended with additional logic for the background saving mechanism. On an interrupt, the register file banks are switched. Parts of the old memory bank (the interrupt context) are copied to the core's stack location, while execution can go ahead by using the new bank. In this design, there is a dedicated memory port for the background saving mechanism, but optionally it can also be shared with the port from the load-store unit. The stack pointer is adjusted appropriately to maintain ABI invariants.

trees. For each interrupt source, they track: (i) interrupt level and priority, (ii) interrupt `id`, and (iii) pending state from the gateway module. The `Interrupt Prioritzation` module traverses the tree from leaves to the root, where the sought-after maximum level and priority interrupt is found. Each tree has low overhead in terms of area and delay, $\mathcal{O}(n)$ and $\mathcal{O}(log(n))$, respectively. The CLIC design proposed in this work can scale up to $n = 4096$ local interrupt sources. Optionally, additional pipeline stages can be inserted in the arbitration tree to relax timing. Finally, our version of the CLIC supports selective hardware vectoring (SHV) and the `xnxti` CSR in the core.

Albeit improving the interrupt handling capabilities of CV32 by introducing priority and levels management in HW, critical operations such as interrupt state and context save/restore are not natively covered by the CLIC and need to be handled in SW. CV32$^{\texttt{fastirq}}$ aims at filling this gap.

C. CV32RT$^{\texttt{fastirq}}$: fast interrupt extension

1) Automatic hardware context saving and bank switching: A block diagram of CV32RT$^{\texttt{fastirq}}$ is shown in Figure 3. Conceptually, we can think of `fastirq` as a wrapper around the core's register file. We extend the register file by an additional read port for the background saving mechanism and registers for latching the additional processor state required for proper interrupt nesting.

A new interrupt at the CLIC will first be checked whether the interrupt level exceeds the configured threshold. If this is the case, it will be dispatched to the core's main state machine while concurrently triggering the saving logic finite state machine (FSM) in the extended register file. The core's state machine will flush the pipeline and update the program counter according to the vector table entry. Meanwhile, in the extended register file, the saving logic triggers a bank switch, allowing the interrupt context to have a fresh set of registers while draining the other bank contents through a separate port to the main memory.

2) Stack Pointer Handling: During a bank switch, we need to update the stack pointer. For that, we have a dedicated adder between the two register files. The RISC-V embedded and integer ABI dictate that the stack pointer points below the last saved register on the stack. Furthermore, if we did not adjust the pointer, the program code running in the interrupt handler could clobber the values on the stack. We could do away with this mechanism for leaf-type interrupts, but this would require a new ABI considering a virtual stack pointer offset when generating code for interrupt handlers, and it would not solve the nested interrupt case.

3) Forwarding Logic: While the interrupt handler can already be executed, we need to ensure that loads or stores access the stack values in memory since these are potentially still being written to memory by the background-saving mechanism. This is handled by informing the load-store unit of the core of the ongoing state being written to memory. A simple solution would be to block any memory accesses while the background saving mechanism is at work. This, though, partially negates the advantage of executing ahead since we likely want to issue a load soon in the interrupt handler. In this implementation, we side-step this problem by checking whether the load-store unit tries to access the memory in the range of the stack pointer. The background saving happens incrementally. After the stack pointer is decremented, making space available, it will start storing the interrupt state word by word. In the load-store unit, the address offset of the last word pushed out by the background saving mechanism is compared against any incoming load and stores. Load and stores that try to access data that is not yet pushed to memory get stalled.

Alternatively, one could forward the value through the load-store unit instead, but this approach is more costly in terms of hardware complexity since this requires a dynamic address lookup into a queue-like buffer.

4) Co-operation with Compiler: Accessing stack memory locations during the execution of an interrupt handler typically happens in two use cases:

- A system call handler (issued through the `ecall` instruction in RISC-V) wants to access user-provided arguments. Most will be passed through general purpose registers, but some might be placed on the stack
- Short interrupt handlers that want to return before the full interrupt state has been saved.

In both cases, we do not need to engage the stalling logic outlined in Section III-C3 by simply ordering the loads to first access the already stored interrupt state. This can be achieved since we know how the interrupt state gets pushed to memory. For example, if we push out the general purpose registers `x1, x2, ...` in that very same order, then we need to ensure that in the interrupt handler, we use the same order to load words back.

Note that this is a performance issue and does not affect correctness. When writing assembly, the programmer has to be aware of that to achieve the best possible latency. In the case that the programmer uses compiler-specific attributes to write his interrupt handlers, such as `__attribute__((interrupt))` in GCC, the compiler needs to be made aware of that fact.

5) Interrupt Handler Routines: In Figure 5, we give an overview of how nested interrupt handling code works for the basic CLINT-mode, the baseline CLIC, and our `fastirq` extension. Due to the inflexible interrupt scheme of the CLINT-mode, much more work needs to be done in managing interrupt mask (`SOME_IRQ_MASK`) and other machine state. The CLIC addresses this with the level/priority scheme and `fastirq` improves upon that by moving the interrupt state saving logic in hardware and adding `emret` to handle redundant interrupt context sequences.

D. Interrupt scenario analysis

1) Non-nested interrupts: As discussed in Section II-B, redundant context restore with non-nested or nested horizontal interrupts can introduce unwanted additional interrupt latency. With the proposed approach, the interrupt state can be quickly restored by simply switching register banks. To differentiate between a regular return from an interrupt handler using `mret`, which assumes the interrupt state has been restored by software, we add a new instruction `emret` instead. This instruction performs the same function as `mret` in addition to switching register banks.

In the RISC-V ecosystem, this situation can be optimized by directly checking for other interrupts pending on the same level before restoring the execution's interrupt context. The CLIC offers a hardware-assisted solution to address such a scenario with the `xnxti` CSRs (Section II-C2). Nuclei's ECLIC [18] extends `xnxti` by embedding the jump to the queuing interrupt handler in the `xnxti` hardware (`jalxnxti`). In our case, this is implemented through the `emret` instruction. Besides checking whether a bank switch is appropriate, it also looks at interrupts pending with the same or lower level. If such an interrupt exists, `emret` redirects the control flow

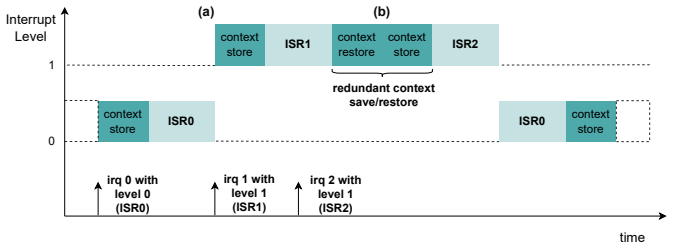


Fig. 4. **(a)** Transition between different interrupt levels. **(b)** Redundant context restore of two non-preemptive interrupts (e.g., two interrupts with same level but different priorities) and tail-chaining to optimize it.

to the pending interrupts handler. Some hardware refers to this concept of removing *redundant context restores* as tail-chaining [23]. This situation is visualized in Figure 4(b).

2) Nested interrupts: Following the notation adopted in Section II-B5, the case of nested interrupts entails preemption of a low-level interrupt by a high-level interrupt, as shown in Figure 4(a). We address this scenario by minimizing *interrupt latency* through hiding latency.

Whenever an interrupt handler is entered, global interrupts are disabled. This increases the worst-case interrupt latency by the time they remain disabled. For nesting interrupts, the first instruction will be re-enabling global interrupts. At this point, a high-level interrupt could preempt the current running interrupt handler. If this happens while the background saving mechanism is still at work, the execution of the handler has to wait.

This combination of latency hiding and background saving allows the core to quickly enter a first-level interrupt handler and potential nesting interrupt handlers, albeit with a larger delay.

Restoring the pre-interrupt context is entirely handled in software for the nested interrupt case. While this return path could also be handled in hardware by adding an additional write port to the core's register file, exiting an interrupt handler is less time-critical since we mostly care about how quickly an external event is addressed.

E. Context Switching acceleration

We can divide context switches into operating system-specific and hardware-specific parts. The operating system part entails all contributions to the context switch time that is specific to the operating system itself, such as computing the next task to be scheduled and bookkeeping operations. The remainder is the hardware-dependent saving and restoring of the state belonging to the new context.

The idea is to use the background saving mechanism to accelerate the state saving and restoring part of context switches by interleaving the loading of a new state with the automatic saving hardware pushing out the previous register state to memory in the background.

For that, the initial part of the context switch routine changes. At the beginning of the routine, we want to save the current running tasks' state to memory. Instead of manually saving the general purpose registers, we set up the stack

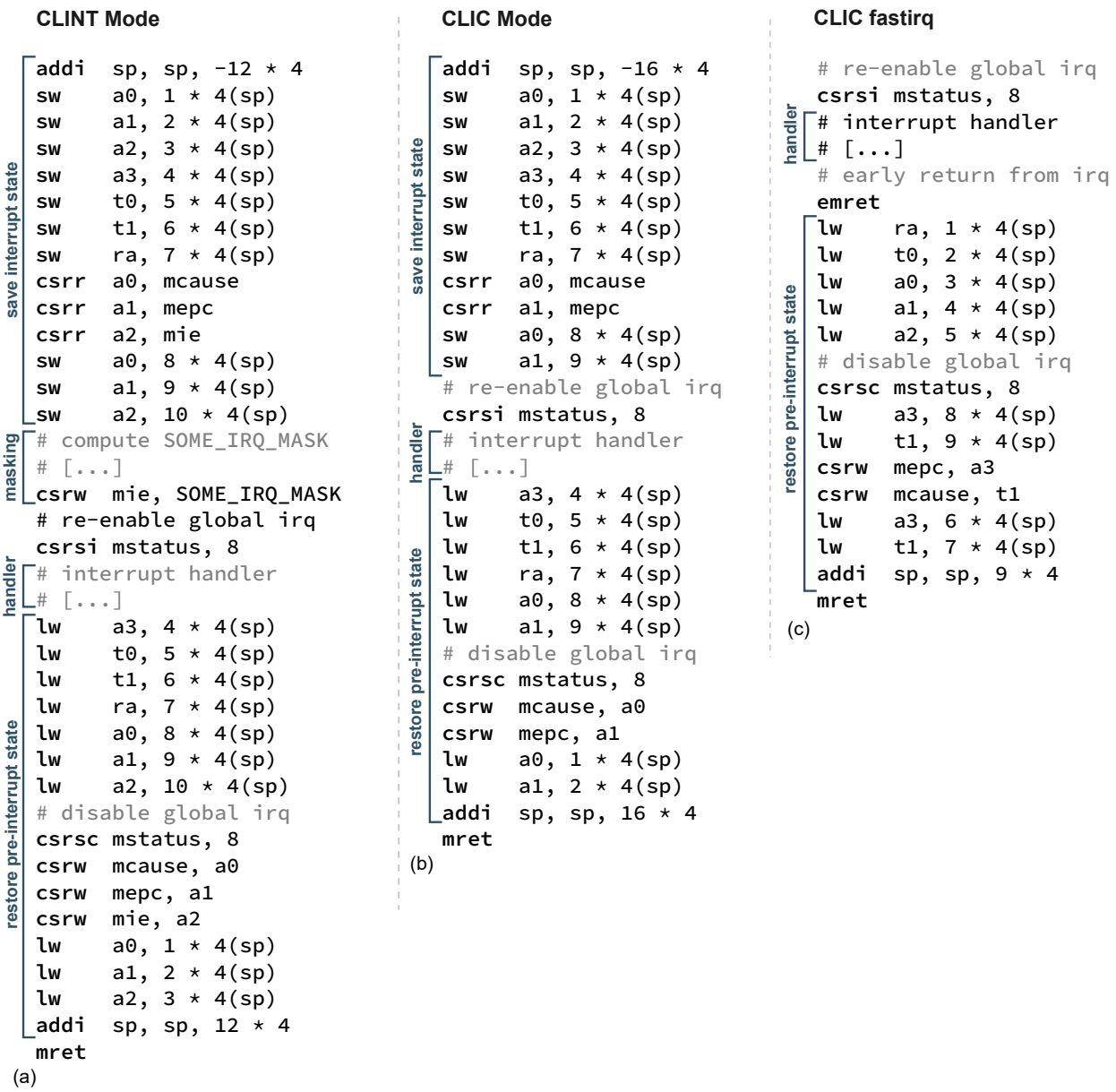


Fig. 5. Routines for saving state for vectored nesting interrupts using the CLINT (a), CLIC (b) and the proposed `fastirq` (c) extension. This assumes the regular RISC-V embedded ABI. Note that in CLINT-mode interrupts with lower priority than the current interrupt running can fire when global interrupts are re-enabled. In order to prevent that, `mie` has to be manually adjusted such that the corresponding lower-priority interrupts are disabled.

pointer appropriately before triggering a software interrupt (write to CLIC's memory map). This will engage the hardware mechanism to swap the registers and save them in the background while we can already proceed with the rest of the context switch routine.

IV. Evaluation

This section gives a functional and quantitative evaluation of the various flavors of the CV32RT. We give an analysis of the functional improvements and argue that these allow more flexible and efficient handling of regular and nested interrupts. Then we show how the various CV32RT versions perform in terms of interrupt latency and context switch times and finally quantify the overhead these additions incur in terms of area and timing.

A. Feature Comparison

As discussed in Section II, the CLINT is the first standardized RISC-V interrupt controller. It supports pre-emption based on privilege modes (machine, hypervisor, supervisor, user), but the interrupt lines have a hardwired prioritization scheme. For the embedded use cases, the CLINT lacks fine-grained control over interrupt prioritization. The baseline CLIC addresses these weaknesses.

Interrupts can be dynamically assigned a priority and a level. The priorities allow concurrent pending interrupts to be taken in the order preferred by the programmer, while the level information enables pre-emption of same-privilege level interrupts (also called horizontal interrupts). Interrupts that are assigned a higher level can pre-empt lower-level interrupts. In addition to that, a level threshold register per privilege

TABLE II
FEATURE COMPARISON OF VARIOUS FLAVORS OF RISC-V CORE-SPECIFIC INTERRUPT CONTROLLERS.

Configuration	Prioritization by *priv.— level — priority*	Vectored Interrupts?	Redundant Context Restore
CV32 (CLINT)	✓— ✗— ✗	Both	✗
CV32RT$^{\texttt{CLIC}}$	✓— ✓— ✓	Both	✗
CV32RT$^{\texttt{CLIC}}$ (`xnxti`)	✓— ✓— ✓	✗	✓
CV32RT$^{\texttt{CLIC}}$ (`jalxnxti`)	✓— ✓— ✓	✗	✓
CV32RT$^{\texttt{fastirq}}$	✓— ✓— ✓	✓	✓

level (`xintthresh`) controls the set of allowed horizontal interrupts by limiting it to those whose's level exceeds the given value in the register. This can be useful in critical sections where only a subset of interrupts need to be disabled.

To improve interrupt latencies, interrupt vectoring is supported. Vectoring can be selectively enabled or disabled per interrupt line. This allows for better control of the vector table size. The baseline CLIC does not differ from the regular CLINT in terms of storing and restoring the interrupt context which is fully handled in software.

The optional `xnxti` extension allows multiple horizontal interrupts to be serviced in sequence without redundant context-restoring operations in between. Reading the `xnxti` CSR yields a pointer to the vector table entry for the next pending and qualifying interrupt, if available, which then allows a direct jump there. This approach loses the latency advantage of hardware vectoring by basically running a software emulation thereof. In addition, the first interrupt has to still pay the full latency cost since `xnxti` does not touch the interrupt context storing part itself.

Our proposed `fastirq` extension addresses these weaknesses by extending the CLIC base capabilities with a mechanism to lower interrupt latency while still keeping hardware vectored interrupts and allowing the skipping of redundant context restore operations.

The discussed differences are summarized in Table II.

B. Functional performance

1) Measurement Setup: We take measurements by running RTL simulations of the different versions of CV32RT as part of ControlPULP. For the memory subsystem, this means we have single cycle (zero wait state) access to static random access memory (SRAM) (Section II-A). The memory bank we are using is not contended by other bus masters which is achieved by placing data and instructions into specific private banks. On the system level, there are no additional latencies introduced on interrupt lines between interrupt sources and the CLIC.

2) Interrupt Latency: We measure the hardware contributed interrupt latency (as described in Section II-B3) of our `fastirq` extension and compare it to the CV32 and CV32RT variations (standard CLIC, `xnxti`, `jalxnxti`). The interrupt latency is measured as the number of cycles it takes for an interrupt to arrive at the interrupt controller input to the first instruction of an interrupt handler that allows the calling of a C-function. This implies that all caller-save registers need to be saved, which is the general case.

Interrupt handler routines that save the interrupt context in software can only save the minimum state if the compiler is able to fully inline the handler's function body. Depending on the function and the ABI, this might be fewer than all caller-save registers. To also consider these cases, we measure the interrupt latency in the optimal case, i.e., only one caller-save register needs saving for software-based interrupt handlers.

Interrupts are injected into the design at the interrupt controller inputs. Each hardware configuration has a handwritten optimized interrupt handler that stores all required general-purpose and machine-specific registers for nesting interrupts. We evaluate both the EABI and regular integer ABI of RISC-V. The saved general-purpose registers are all caller-save registers in the respective ABI. This allows for directly calling into C-code functions from interrupt handlers. For our `fastirq` extension, this amount of state is always saved since the hardware does not know a priori which caller-save registers need to be saved. Interrupt handler routines that use software-based mechanisms to save and restore interrupt state can, if the compiler permits, fully inline the handler code and save some caller-save registers.

The results are summarized in Figure 6. Compared to the baseline CLINT and CLIC which both have about 33 cycles interrupt latency, the `fastirq` extension is able to reduce this down to 6 cycles. `xnxti` and `jalxnxti` perform even worse (requiring 42 and 35 cycles, respectively) since they both insert additional instructions in the code path between the handler and interrupt event. As a comparison, the Arm Cortex-M4 has an interrupt latency of 12 cycles given a single-cycle memory [24].

3) Redundant Context Restoring: As described in Section II-B, whenever there are *back-to-back* interrupts, the interrupt context restore and store sequence between them can be considered redundant.

In Figure 6 we show the cost in clock cycles of such sequences. For the baseline CLIC it takes 68 or 50 cycles when using the integer or embedded ABI respectively. In this case, the redundant context restoring sequences contains the full interrupt exit code sequence. The latter consists of restoring the interrupt context and the regular interrupt latency. `xnxti` and `jalxnxti` improve upon this situation by checking for pending interrupts and directly jumping to the respective handlers. Note that this is independent of the ABI since no interrupt state is affected. While `xnxti` only returns a pointer to the address of the next handler and thus needs a small code sequence (load, jump, and retry loop), `jalxnxti` fuses these operations into one instruction resulting in saving 9 cycles. The `emret` mechanism of `fastirq` works similarly and costs 8 clock cycles but is not restricted to non-vectored interrupts. The Arm Cortex-M4 is able to do the same task in 6 cycles assuming it has access to single-cycle memory [24].

4) Context Switch Time: In Figure 7 we show the context switch time in number of clock cycles between two FreeRTOS dummy tasks when comparing the baseline CV32RT against CV32RT$^{\texttt{fastirq}}$. All compile time options such as tracing, stack overflow signaling, and the more generic task selection mechanism were turned off to minimize the context switch code. We left out configurations that do not have any impact

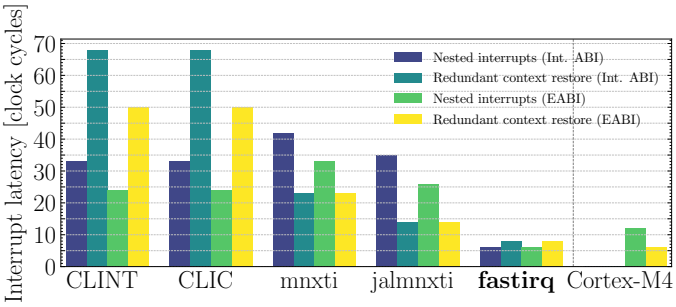


Fig. 6. Interrupt latency of various flavors of RISC-V core-specific interrupt controllers for interrupt handlers that support nesting and the calling of C-functions within it (i.e., saving/restoring state following the C-ABI). As a comparison, we mention the generic Cortex-M4 core, assuming it has single-cycle access to memory. Note that Arm Cortex-M only has 16 core registers.

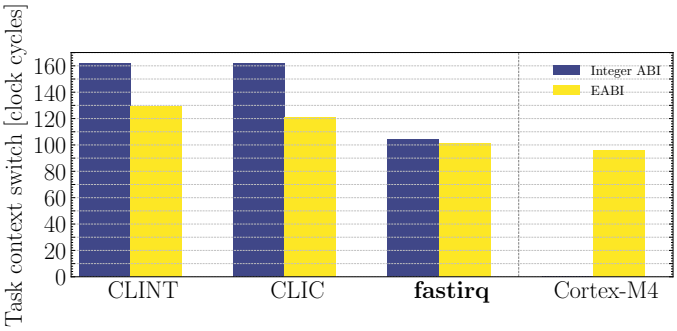


Fig. 7. Average Context Switch Time in FreeRTOS for two tasks of various flavors of CV32RT. The RISC-V E-Extension reduces the available general purpose registers from 32 to 16, consequently lowering the context switch state that needs to be saved and restored.

on context switching. `fastirq` allows us to skip ahead the saving of the general purpose registers by triggering a software interrupt as part of the save sequence. The software interrupt will trigger the `fastirq` mechanism, which starts saving the general purpose registers to memory. By making sure not to use registers that are still being saved by the background saving mechanism (as described in Section III-C4) we can save up to 31 cycles (-19%) for a context switch when using the I-extension, and 16 cycles (-12%) for the E-extension, respectively. The FreeRTOS website [20] claims context switches as low as 96 cycles for a Cortex-M4 implementation. [3]

C. Implementation

We synthesize CV32RT as part of the ControlPULP platform using Synopsys Design Compiler 2022.03, targeting GlobalFoundries 12LP FinFet technology at 500 MHz, TT corner, and 25 °C. One gate equivalent (GE) for this technology equals 0.121 μm^2.

Figure 8 reports the area breakdown of the CLIC implemented in the proposed with different interrupt sources. As the figure shows, for each scenario, more than half of the resources implement the configuration registers required to control the CLIC, which size linearly increases with the

[3]This can be seen as a lower bound as on some SoCs such as the STM32L476RG we observed higher latencies due to memory access stalls and other implementation choices in the memory subsystem.

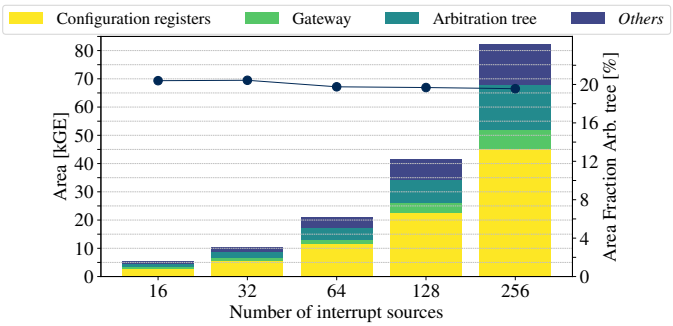


Fig. 8. RISC-V CLIC area breakdown at varying numbers of interrupt sources.

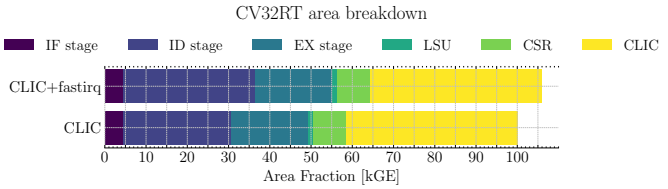


Fig. 9. Area overhead of CV32RT in the two main configurations. The overhead of `fastirq` in CV32RT$^{\texttt{fastirq}}$ core results in a minimal 10% area increase concentrated around the ID stage. The design has been synthesized in GF12LP technology (500 MHz, *TT* corner, 25 °C, 0.8V, super low V_T standard cells).

number of input interrupts. In particular, each interrupt is associated with one 32-bit register incurring an area overhead of about 176 GE. The remaining area implements the gateway and binary tree arbitration logic at the core of the CLIC working principle, as discussed in Section III-B, and additional housekeeping control logic that scales linearly with the number of interrupt sources. Figure 8 also shows that the fraction of the design occupied by the arbitration tree is kept constant when increasing the number of sources. While it is apparent that this design incurs a larger area overhead compared to traditional RISC-V CLINT [14], the gain in flexibility enables a broader application scope with time-critical systems.

Figure 9 reports the area breakdown comparison of CV32RT with baseline CLIC (including `mnxti` and `jalmnxti` CSRs) and `fastirq` extensions with 256 input interrupts. From the table, CV32RT$^{\texttt{fastirq}}$ incurs a 10% overall area increase compared to baseline CLIC only. While other HW blocks of the core remain primarily unaffected, the instruction decode (ID) stage incurs an area overhead of 21% compared to CV32RT$^{\texttt{CLIC}}$. The ID stage is the HW block where the additional registers and the automatic stacking/unstacking logic are localized. A breakdown of the ID stage is shown in Figure 10. We notice that the additional storage space for automatic context save and restore in HW increases the area of the register file by about 36% in the proposed implementation. Similarly, the logic for managing the shadow registers accounts for an overhead of 40% on the baseline *ID stage* controller. The HW overhead coming from the additional `emret` instruction discussed in Section IV is negligible. Nevertheless, the increased size of the ID stage trades off the benefits of (i) a simplified programming model that moves several SW operations in HW, and (ii) significantly lowered interrupt latency than standard RISC-V while not impacting the critical path of the base core

TABLE III
COMPARISON OF THE MAIN TECHNIQUES FOR OPTIMIZING INTERRUPT CONTEXT AND TASK CONTEXT SAVE/RESTORE WITH NESTED AND NON-NESTED INTERRUPTS EMPLOYED BY INDUSTRY AND ACADEMIA IN THE EMBEDDED AND REAL-TIME APPLICATION DOMAINS.

Processor	Interrupt controller	ISA	Interrupt context save/restore acceleration	Task context save/restore acceleration	Back-to-back interrupts acceleration	Open?	Max. #levels	Interrupt latency	Task context switch time
Industry									
Arm Cortex M4 [20], [25]	NVIC	Arm RISC	Automatic in HW	n.a.	NVIC tail-chaining	✗	256	12	96
Arm Cortex R5	VIC/GIC	Arm RISC	n.a.	Register banking	n.a.	✗	n.a.	20	n.a.
Infineon AURIX	ICU	TriCore RISC	Automatic in HW	SW based - context save area (CSA)	n.a.	✗	256	10-16	156-162
Renesas M32C/80	IC	M32C/80	Dual register bank	Dual register bank	n.a.	✗	8	5 [a]	n.a.
Nuclei System Technology	ECLIC	RISC-V	n.a.	n.a.	`jalmnxti`	✗	256	4-6 [b]	n.a.
Alibaba's T-Head Xuantie E906	CLIC	RISC-V	n.a.	n.a.	`mnxti`	✓	n.a.	n.a.	n.a.
SiFive E21	CLIC	RISC-V	n.a.	n.a.	`mnxti`	✗	16	20 [c]	n.a.
ESP32 C3	INTC	RISC-V	n.a.	n.a.	n.a.	✗	15	n.a.	n.a.
-	AIA local	RISC-V	-	-	`xtopi`	✗	-	-	-
Academia									
Gaitan et al. [26]	MPRA/HSE	RISC	Automatic in HW	Automatic in HW	n.a.	✗	n.a.	1.5 [d]	3 [d]
Mao et al. [27]	CLIC	RISC-V	Automatic in HW	n.a.	n.a. [e]	✗	n.a.	13	n.a.
Balas et al. [28]	CLINT	RISC-V	-	-	-	✓	-	24	129
This work	**CLIC**	**RISC-V**	**Hybrid [f]**	**Hybrid**	**`emret`**	✓	**256**	**6**	**114**

[a] Single high-speed interrupt. Refers to the number of clock cycles from the firing of an interrupt to the execution of the interrupt body. Does not count the saving of the general purpose registers. [29].
[b] Refers to the number of cycles from arrival the arrival of the interrupt to the first instruction fetch of the ISR [30]. Does not count the saving of the interrupt context, including the general purpose registers.
[c] Refers to the number of the clock cycles it takes from the firing of an external interrupt to the time the first instruction in the corresponding interrupt service routine of a C function is executed [18].
[d] Full HW solution. SW scheduling latency becomes negligible as from Figure 1, which justifies the performances in the order of the clock cycle.
[e] Implementation details are omitted.
[f] Combination of background saving/restoring and register banking.

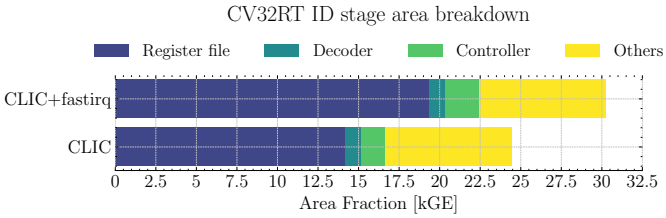


Fig. 10. Area breakdown of CV32E40P^{RT} *ID stage* with the proposed hardware extensions.

design. Lastly, time-critical systems shift design priorities from area efficiency to safety, security, and reliability.

V. RELATED WORK

In this section, we describe the leading solutions to optimize handling asynchronous events in state of the art (SOTA) embedded and real-time MCUs. We first differentiate between existing platform-level and core-local interrupt controllers as introduced in Section II-C. Then, we focus on the latter and discuss solutions across various platforms in industry and academia, addressing (i) interrupt context save/restore techniques, (ii) context switch techniques, and (iii) dedicated strategies to optimize redundant context restore with back-to-back interrupts. Table III summarizes the overview. For the interrupt latency, we assume the definition presented in Section II-B and explicitly provide references to different variants adopted by SOTA when needed in the table.

A. Platform-level interrupt controllers

Controllers that belong to this category, such as the RISC-V PLIC and Advanced Platform-Level Interrupt Controller (APLIC) for wire-based interrupt communication and RISC-V incoming message signaled interrupt controller (IMSIC) for message-signaled interrupt communication, are not designed to distribute time-critical interrupts to the running HARTs. Conversely, Arm's generic interrupt controller (GIC) redistributes incoming asynchronous events as either non-critical (`IRQ`) or critical interrupts (fast `IRQ`, or `FIQ`). The latter reduces the execution time of the interrupt handler through a dedicated register bank, with up to eight registers employed to minimize context switching.

B. Core-local interrupt controllers

Core-local interrupt controllers are often specialized in providing fast interrupt-handling capabilities in real-time embedded application domains.

1) Interrupt context save/restore acceleration: Several designs automatically save and restore the interrupt context directly in HW. This allows the core to simultaneously start fetching the jump address from the interrupt vector table during context save and embed context restore within the return instruction. A reference example in the field is the Arm Cortex-M series integrating the nested vectored interrupt controller (NVIC). It implements a state machine [31] that performs caller-save register stacking in the background. Likewise, upon returning from the ISR, the HW encodes in the link register a value (`EXC_RETURN`), which notifies the core to start unwinding the stack to return to normal program execution. Analogously, in Infineon AURIX MCU-class TriCore family's interrupt control unit (ICU) [32]–[34] the context of the calling routine is saved in memory autonomously while restoring the context is embedded in the `RET` instruction and happens in parallel with the return jump [35]. From academia, Mao et al. [27] are the first to propose extensions for the RISC-V CLIC. Though implementation details are omitted, the interrupt handling is enhanced with automatic stacking in hardware that benefits from the core's Harvard architecture and simultaneous data and instruction memory access.

Albeit automatic interrupt context save/restore reduces SW housekeeping overhead before and after handling the interrupt routine, it only partially addresses the acceleration of the complete task context switch.

2) Task context save/restore acceleration: Register banking is a technique adopted by several interrupt controller architectures to swap a task's context without pushing/popping register values to the stack at the cost of an additional area overhead in the design. As discussed above, in Arm designs, this is the case of HW register banking in platform-level interrupt controllers (GIC). A similar approach is implemented in the Renesas M32C/80 series [29]. A dual register bank allows quickly swapping the context without saving/restoring to/from the stack, as the second register bank is reserved for high-speed interrupts. The Aurix family instead implements a SW managed solution, featuring a specific organization of the context layout in the system memory based on CSAs chained in a linked list fashion [35]. These approaches trade-off HW and SW-induced latencies when handling asynchronous events, as depicted in Figure 1. Gaitan et al. [26] propose instead a full HW solution based on a Hardware Scheduling Engine (HSE) that directly attaches interrupts to running tasks without the need for a specialized interrupt controller. While this approach allows lowering interrupt latency and task context switches dramatically, as reported in Table III, it is expensive in terms of area as it requires replicating hardware resources per task.

3) Optimizations for non-nested interrupt handling: Redundant context restore with non-nested interrupts is addressed by chaining two back-to-back interrupts and bypassing the superfluous restore/save operation. RISC-V addresses this scenario by adding the `xnxti` CSR with SW-managed interrupt service loops as part of the CLIC specifications and finds early commercial examples with [30]. Nuclei System Technology *enhanced CLIC* (ECLIC) [36] extends traditional `xnxti` with a novel CSR for machine privilege mode, `jalmnxti` [18], already discussed in this work in Section IV. Among MSI solutions, RISC-V AIA without APLIC has a similar approach to `mnxti` with the `xtopi` CSR that reports the highest-priority, pending, and enabled interrupt for a specific privilege mode, allowing both late arrival and redundant context restore mechanisms.

While the presented solutions are effective in optimizing context, save/restore with HW and SW cooperation, they lack (i) a cohesive approach to address both interrupt context and task context switch acceleration and (ii) none of the existing RISC-V-based approaches can close the gap with well-established industry vendors.

This work proposes a combination of the background-saving with a register banking approach. Interrupt context save/restore especially takes advantage of the former by deferring those operations to the HW. This allows execution of the interrupt handler before all interrupt state has been pushed to memory, lowering interrupt latency. A task's context switch benefits from quickly transferring the suspended context to the dedicated register bank while already restoring the next task to be executed.

To address redundant context restore with back-to-back interrupts, we propose a modified version of the return instruction in machine mode (i.e., `emret`), which is able to skip redundant context saving and restoring sequences by directly jumping to the next available interrupt handler. This solution works for both vectored and non-vectored interrupts.

VI. Conclusion

In this work, we present `fastirq`, a fast interrupt extension for RISC-V embedded systems. We implement the extension on CV32RT, a 32-bit, in-order, single-issue core designed with the RISC-V CLIC fast interrupt controller. With our design, we can achieve interrupt latencies of 6 clock cycles and efficient back-to-back interrupt handling in 12 cycles which is as low as the fastest available approaches currently implemented in the RISC-V landscape, fully open-source, and competitive against closed-source and proprietary commercial solutions. Furthermore, we improve task context switch times in FreeRTOS to 104 clock cycles using `fastirq`, which is 20% faster than a software-only approach.

References

[1] C. Rochange, S. Uhrig, and P. Sainrat, *Time-Predictable Architectures*, ser. FOCUS - Computer Engineering Series. iSTE / Wiley, 2014. [Online]. Available: http://eu.wiley.com/WileyCDA/WileyTitle/productCd-1848215932.html

[2] L. M. Pinho, E. Quinones, M. Bertogna, A. Marongiu, V. Nelis, P. Gai, and J. Sancho, *High-Performance and Time-Predictable Embedded Computing*. Wharton, TX, USA: River Publishers, 2018.

[3] F. Reghenzani, G. Massari, and W. Fornaciari, "The Real-Time Linux Kernel: A Survey on PREEMPT_RT," *ACM Comput. Surv.*, vol. 52, DOI 10.1145/3297714, no. 1, Feb. 2019. [Online]. Available: https://doi.org/10.1145/3297714

[4] M. Liu, D. Liu, Y. Wang, M. Wang, and Z. Shao, "On improving real-time interrupt latencies of hybrid operating systems with two-level hardware interrupts," *IEEE Transactions on Computers*, vol. 60, DOI 10.1109/TC.2010.119, no. 7, pp. 978–991, 2011.

[5] P. Mantegazza, E. L. Dozio, and S. Papacharalambous, "Rtai: Real time application interface," *Linux J.*, vol. 2000, no. 72es, p. 10–es, Apr. 2000.

[6] K. Ramamritham and J. Stankovic, "Scheduling algorithms and operating systems support for real-time systems," *Proc. IEEE*, vol. 82, DOI 10.1109/5.259426, no. 1, pp. 55–67, 1994.

[7] D. Kleidermacher, "Minimizing interrupt response time," 2005. [Online]. Available: http://web.engr.oregonstate.edu/~traylor/ece473/pdfs/minimize_interrupt_response_time.pdf

[8] J. Valvano, *Introduction to Embedded Systems*, 08 2016.

[9] Y. Huang, L. Shi, J. Li, Q. Li, and C. J. Xue, "Wcet-aware re-scheduling register allocation for real-time embedded systems with clustered vliw architecture," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 22, DOI 10.1109/TVLSI.2012.2236114, no. 1, pp. 168–180, 2014.

[10] X. Zhou and P. Petrov, "Rapid and low-cost context-switch through embedded processor customization for real-time and control applications," in *Proceedings of the 43rd Annual Design Automation Conference*, ser. DAC '06, DOI 10.1145/1146909.1147001, p. 352–357. New York, NY, USA: Association for Computing Machinery, 2006. [Online]. Available: https://doi.org/10.1145/1146909.1147001

[11] I. Behnke, L. Pirl, L. Thamsen, R. Danicki, A. Polze, and O. Kao, "Interrupting real-time IoT tasks: How bad can it be to connect your critical embedded system to the internet?" in *2020 IEEE 39th International Performance Computing and Communications Conference (IPCCC)*, DOI 10.1109/ipccc50635.2020.9391536. IEEE, Nov. 2020. [Online]. Available: https://doi.org/10.1109%2Fipccc50635.2020.9391536

[12] F. Rehm, J. Seitter, J.-P. Larsson, S. Saidi, G. Stea, R. Zippo, D. Ziegenbein, M. Andreozzi, and A. Hamann, "The road towards predictable automotive high - performance platforms," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, DOI 10.23919/DATE51398.2021.9473996, pp. 1915–1924, 2021.

[13] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for risc-v," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug. 2014. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[14] A. Waterman, Y. Lee, R. Avizienis, D. A. Patterson, and K. Asanović, "The risc-v instruction set manual volume ii: Privileged architecture version 1.9," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-129, Jul. 2016. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-129.html

[15] RISC-V, ""smclic" core-local interrupt controller (clic) risc-v privileged architecture extension." [Online]. Available: https://github.com/riscv/riscv-fast-interrupt/blob/master/clic.adoc

[16] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini, "Near-threshold risc-v core with dsp extensions for scalable iot endpoint devices," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, DOI 10.1109/TVLSI.2017.2654506, no. 10, pp. 2700–2713, 2017.

[17] D. Schiavone, *OpenHW Group CV32E40P User Manual*, OpenHW Group. [Online]. Available: https://cv32e40p.readthedocs.io/en/latest/

[18] N. S. Technology, *Nuclei ISA spec*, Nuclei System Technology, 2021. [Online]. Available: https://doc.nucleisys.com/nuclei_spec/isa/introduction.html

[19] A. Ottaviano, R. Balas, G. Bambini, A. del Vecchio, M. Ciani, D. Rossi, L. Benini, and A. Bartolini, "Controlpulp: A risc-v on-chip parallel power controller for many-core hpc processors with fpga-based hardware-in-the-loop power and thermal emulation," *ArXiv*, vol. abs/2306.09501, 2023.

[20] R. Barry. Freertos: Real-time operating system for microcontrollers. Real Time Engineers Ltd. (2023). [Online]. Available: https://www.freertos.org/index.html

[21] C.-M. Lin, "Nested interrupt analysis of low cost and high performance embedded systems using gspn framework," *IEICE Transactions*, vol. 93-D, DOI 10.1587/transinf.E93.D.2509, pp. 2509–2519, 09 2010.

[22] R.-V. T. Group, *RISC-V Platform-Level Interrupt Controller Specification*, RISC-V International, 2023. [Online]. Available: https://github.com/riscv/riscv-plic-spec/blob/master/riscv-plic-1.0.0.pdf

[23] J. Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, Third Edition*, 3rd ed. USA: Newnes, 2013.

[24] Arm, *Cortex-M4 Technical Reference Manual*, Arm, 2020. [Online]. Available: https://developer.arm.com/documentation/100166/0001/

[25] J. Yiu, *A Beginner's Guide on Interrupt Latency - and Interrupt Latency of the Arm Cortex-M processors*, Arm, 2012. [Online]. Available: https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/beginner-guide-on-interrupt-latency-and-interrupt-latency-of-the-arm-cortex-m-processors?pifragment-22714=2

[26] V. G. Gaitan, N. C. Gaitan, and I. Ungurean, "Cpu architecture based on a hardware scheduler and independent pipeline registers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 23, DOI 10.1109/TVLSI.2014.2346542, no. 9, pp. 1661–1674, 2015.

[27] B. Mao, N. Tan, T. Chong, and L. Li, "A clic extension based fast interrupt system for embedded risc-v processors," in *2021 6th International Conference on Integrated Circuits and Microsystems (ICICM)*, DOI 10.1109/ICICM54364.2021.9660345, pp. 109–113, 2021.

[28] R. Balas and L. Benini, "Risc-v for real-time mcus - software optimization and microarchitectural gap analysis," in *2021 Design, Automation, Test in Europe Conference Exhibition (DATE)*, DOI 10.23919/DATE51398.2021.9474114, pp. 874–877, 2021.

[29] Renesas, *Hardware Manual - RENESAS MCU M16C FAMILY / M32C/80 SERIES*, Renesas, 2023. [Online]. Available: https://www.renesas.com/us/en/document/mah/m32c87-group-m32c87-m32c87a-m32c87b-hardware-manual

[30] S. Inc., *SiFive E21 Core Complex Manual*, SiFive Inc., 2021. [Online]. Available: https://sifive.cdn.prismic.io/sifive/7c22c2ec-8af4-4b6c-a5fe-9327d91e7808_e21_core_complex_manual_21G1.pdf

[31] STMicroelectronics, *STM32L5 - NVIC*, STMicroelectronics, 2020. [Online]. Available: https://www.st.com/content/ccc/resource/training/technical/product_training/group1/61/35/d2/07/34/6f/4e/83/STM32L5-System-Nested_Vectored_Interrupt_Control_NVIC/files/STM32L5-System-Nested_Vectored_Interrupt_Control_NVIC.pdf/_jcr_content/translations/en.STM32L5-System-Nested_Vectored_Interrupt_Control_NVIC.pdf

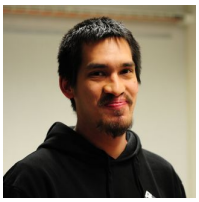
[32] H. C. . Infineon Technologies AG, "A fast powertrain microcontroller." [Online]. Available: https://old.hotchips.org/wp-content/uploads/hc_archives/hc16/3_Tue/6_HC16_Sess7_Pres1_bw.pdf

[33] I. T. AG, *TC27x D-Step, 32-Bit Single-Chip Microcontroller*, Infineon Technologies AG, 2014. [Online]. Available: https://hitex.co.uk/fileadmin/uk-files/downloads/ShieldBuddy/tc27xD_um_v2.2.pdf

[34] I. T. AG, *TriCore V1.6, Core architecture*, Infineon Technologies AG, 2012. [Online]. Available: https://www.infineon.com/dgdl/tc1_6__architecture_vol1.pdf?fileId=db3a3043372d5cc801373b0f374d5d67

[35] I. T. AG, "Task context switching rtos," U.S. Patent US7434222B2, Oct. 2008.

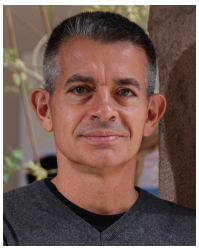
[36] N. S. T. C. Ltd., *ECLIC Unit Introduction*, Nuclei System Technology Co. Ltd. [Online]. Available: https://doc.nucleisys.com/nuclei_spec/isa/eclic.html



Robert Balas received his BSc and MSc degrees in electrical engineering and information technology from ETH Zurich in 2015 and 2017, respectively. He is currently pursuing a Ph.D. degree in the Digital Circuits and Systems group of Prof. Benini. His research interests include real-time computing, compilers, and operating-systems.



Alessandro Ottaviano received the B.Sc. in Physical Engineering from Politecnico di Torino, Italy, and the M.Sc. in Electrical Engineering as a joint degree between Politecnico di Torino, Grenoble INP-Phelma and EPFL Lausanne, in 2018 and 2020 respectively. He is currently pursuing a Ph.D. degree in the Digital Circuits and Systems group of Prof. Benini. His research interests include real-time computing, power management of HPC processors, and energy-efficient processor architecture.



Luca Benini holds the chair of digital Circuits and systems at ETHZ and is a Full Professor at the Università di Bologna. He received a Ph.D. from Stanford University. He is a Fellow of the ACM and a member of the Academia Europaea. He is the recipient of the 2023 IEEE CS E.J. McCluskey Award.